

Universidade Estadual de Mato Grosso do Sul

Curso de Ciência da Computação

Disciplina de Redes de Computadores

Trabalho de Redes de Computadores

PLATAFORMA DE MONITORAMENTO INTELIGENTE DE FILAS
EM TEMPO REAL

Documentação

Victor Vendruscolo

Dourados (MS), 29 de julho de 2026.

Contents

1	Sumário do problema	1
2	Algoritmos, tipos de dados, funções e decisões	1
2.1	Arquitetura geral	1
2.2	Tipo abstrato de dados: a fila	2
2.3	Protocolo de mensagens	3
2.4	Módulos e funções	3
2.5	Algoritmos principais	4
2.6	Decisões de implementação	5
3	Decisões de implementação omissas na especificação	6
3.1	Elementos acrescentados por necessidade do protocolo	6
3.2	Pontos ambíguos e como foram resolvidos	7
3.3	Escopo declarado: o que não foi implementado	8
3.4	Limitação conhecida: estouro de inteiro no identificador	8
3.5	Persistência em arquivos de texto	8
4	Retransmissão de mensagens	9
4.1	O que precisava ser garantido	9
4.2	Mecanismo adotado	9
4.3	Avaliação honesta do mecanismo	10
5	Testes e análise	10
5.1	Como os testes foram feitos	10
5.2	Teste funcional	10
5.3	Casos especiais: 30 de 30 aprovados	11
5.4	Retransmissão vista do lado do cliente	12
5.5	Compilação	13
6	Teste de carga com 100, 1.000 e 10.000 clientes	13
6.1	Como a carga foi gerada	13
6.2	Resultados	13
6.3	Análise dos resultados	14
7	Telas do funcionamento correto	15
8	Conclusão	17
	Referências bibliográficas	18

1 Sumário do problema

Este trabalho implementa uma plataforma cliente-servidor para monitorar uma fila de atendimento em tempo real. Um servidor central mantém a fila; vários clientes se conectam ao mesmo tempo e operam sobre ela. O sistema foi escrito em C, sobre sockets TCP, e roda em Linux.

O problema tratado é a falta de informação compartilhada em locais de atendimento ao público. Quando cada posto anota a fila por conta própria, ninguém sabe o estado real do atendimento. O enunciado cita superlotação, filas desorganizadas e ausência de atualização em tempo real como sintomas disso.

A solução adotada centraliza a fila em um único servidor. Todo cliente que cadastra um usuário altera a mesma estrutura, e todos os demais são avisados na hora. Assim, o estado da fila é único e visível para todos os operadores conectados.

O sistema oferece quatro operações, que reproduzem as telas do enunciado:

- **Autenticação:** o cliente se identifica ao conectar, com uma credencial fixa definida no código.
- **Adicionar usuário:** o operador informa um identificador e um nome, que entram na fila compartilhada.
- **Ver fila:** o servidor devolve a fila completa, do primeiro ao último registro.
- **Heartbeat:** o servidor devolve apenas os usuários cadastrados por outros clientes desde a última verificação. Se não houver novidade, devolve ALIVE.

Além dessas quatro operações pedidas, o servidor envia uma notificação espontânea. Sempre que alguém cadastra um usuário, todos os outros clientes recebem a mensagem de *broadcast* sem terem pedido nada. É esse mecanismo que caracteriza o tempo real do sistema.

Terminologia. As telas do enunciado usam a palavra “paciente”, mas o texto do enunciado usa “usuário”. Este documento adota **usuário** para quem entra na fila e **cliente** para o programa ./cliente que se conecta e opera o sistema. São dois conceitos diferentes e a distinção vale para todo o documento.

2 Algoritmos, tipos de dados, funções e decisões

2.1 Arquitetura geral

O sistema tem dois programas, ligados por uma conexão TCP na porta 8080. A persistência fica em arquivos de texto, do lado do servidor.

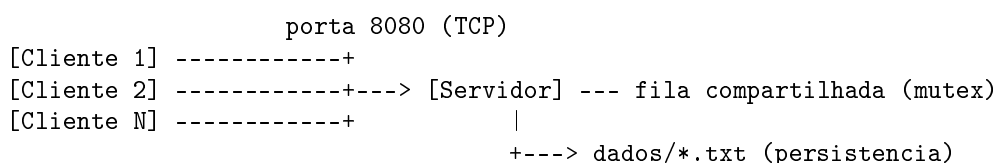


Figure 1: Arquitetura do sistema, correspondente ao diagrama do enunciado.

A técnica de múltiplos clientes escolhida foi **multithreads** (pthreads), uma das três permitidas pelo enunciado. O servidor cria uma thread para cada conexão aceita. Duas razões pesaram nessa escolha. A primeira é que a fila é compartilhada por todos os clientes: com threads, todas enxergam a mesma memória, e basta um *mutex* para proteger o acesso. A segunda é que o *broadcast* precisa alcançar sockets de outras conexões, o que também sai de graça no mesmo processo. Com `fork()`, as duas coisas exigiriam comunicação entre processos.

A thread principal do servidor não atende nenhum cliente. Ela apenas aceita conexões, cria a thread correspondente e volta a aceitar. Isso atende literalmente ao enunciado, que reserva ao servidor principal apenas aceitar conexões, distribuir eventos e monitorar sockets.

```
for (;;) {
    int sock_cliente = accept(sock_escuta, ...);
    ...
    ArgumentoThread *arg = malloc(sizeof(ArgumentoThread));
    arg->sock = sock_cliente;
    arg->numero = ++total_conexoes;
    pthread_create(&thread, &atributos, atende_cliente, arg);
}
```

Listing 1: Laço principal do servidor, em `servidor.c`. A thread criada recebe uma struct própria, com o socket, o IP de origem e o número da conexão.

2.2 Tipo abstrato de dados: a fila

A fila é o dado central do sistema. Ela é um vetor de tamanho fixo, protegido por um único *mutex*, e todo acesso passa pelas funções do módulo `fila.c`. Manter o acesso concentrado em um módulo torna impossível esquecer de travar.

```
typedef struct {
    int id; /* identificador digitado pelo
    operador */
    char nome[TAM_NOME]; /* nome, terminado em '\0'
    */
} Usuario;

static Usuario fila[MAX_USUARIOS_FILA]; /* 20000
    registros */
static int total; /* posicoes em
    uso */
static pthread_mutex_t mutex; /* protege os
    dois */
```

Listing 2: Tipo `Usuario` e estado interno da fila.

O vetor foi preferido à lista ligada pela simplicidade. O limite de 20.000 registros cobre com folga o teste de carga de 10.000 clientes, mesmo que cada um insira mais de um usuário. Esse limite é uma decisão de implementação declarada na Seção 3.

Cada conexão guarda, por conta própria, um marcador do último índice da fila que já viu. É esse marcador que permite ao *heartbeat* devolver apenas o que é novo para aquele cliente.

2.3 Protocolo de mensagens

Toda mensagem é uma linha de texto ASCII terminada em `\n`. O comando vem em maiúsculas, seguido dos argumentos separados por espaço. Essa escolha resolve o problema central de trabalhar sobre TCP: o TCP entrega um fluxo de bytes, não mensagens delimitadas, e uma chamada de `send()` não corresponde necessariamente a uma de `recv()`. Quem lê acumula os bytes em um buffer próprio até encontrar a quebra de linha.

Respostas de várias linhas usam um cabeçalho e um rodapé fixos. O cliente lê linha a linha até bater no rodapé, sem precisar saber quantas linhas virão. É o formato que o próprio enunciado mostra nas telas.

Table 1: Mensagens do cliente para o servidor.

Operação	Formato	Quando
Login	LOGIN <usuario> <senha>	Primeira mensagem da sessão
Adicionar usuário	ADD <seq> <id> <nome>	Opção 1 do menu
Ver fila	LIST <seq>	Opção 2 do menu
<i>Heartbeat</i>	HEARTBEAT <seq>	Opção 3 do menu
Sair	SAIR	Opção 0 do menu

Table 2: Mensagens do servidor para o cliente.

Resposta	Formato	Quando
Login aceito	LOGIN_OK	A credencial confere
Login recusado	LOGIN_FAIL	A credencial não confere; o servidor fecha a conexão
Usuário inserido	ADD_OK <id> <nome>	Após gravar na fila e disparar o <i>broadcast</i>
Fila	==== FILA ===== <id> - <nome> × N =====	Resposta de LIST
<i>Heartbeat</i> com novidade	Mesmo formato da fila, só com os registros de outros clientes	Resposta de HEARTBEAT
<i>Heartbeat</i> sem novidade	ALIVE	Resposta de HEARTBEAT
Erro	ERRO <descrição>	Comando inválido; a conexão continua aberta
<i>Broadcast</i>	[Broadcast] Novo usuario: <nome>	Mensagem não solicitada, enviada a todos os clientes menos ao autor do ADD

O número de sequência <seq> acompanha todo comando posterior ao login. Ele começa em 1 e aumenta a cada novo comando daquela conexão. Sua função é sustentar a retransmissão, tratada na Seção 4.

2.4 Módulos e funções

O código está dividido em seis módulos, além dos dois programas principais e do gerador de carga. Cada arquivo tem uma responsabilidade só.

Table 3: Módulos do sistema e suas funções principais.

Arquivo	Responsabilidade	Funções principais
comum.h	Constantes, tipo <code>Usuario</code> e textos do protocolo	Somente definições
protocolo.c/.h	Envio e leitura de mensagens delimitadas por linha	protocolo_leitor_init, protocolo_le_linha, protocolo_envia_linha, protocolo_limpa_bordas
fila.c/.h	Fila compartilhada protegida por <i>mutex</i>	fila_init, fila_adiciona, fila_tamanho, fila_copia_intervalo
sessoes.c/.h	Registro dos clientes conectados e envio do <i>broadcast</i>	sessoes_init, sessoes_registra, sessoes_remove, sessoes_broadcast, sessoes_trava_envio, sessoes-libera_envio
persistencia.c/.h	Gravação nos quatro arquivos de texto	persistencia_init, persistencia_log_servidor, persistencia_log_sessao, persistencia_historico_add, persistencia_salva_fila
servidor.c	Aceitação de conexões e atendimento	main, atende_cliente, autentica, trata_add, trata_list, trata_heartbeat, envia_bloco_fila, envia_resposta, verifica_sequencia, guarda_resposta_add, nome_valido
cliente.c	Interface de operação	main, conecta_ao_servidor, autentica, thread_receptora, envia_comando, imprime_resposta, opcao_adicionar_usuario, opcao_ver_fila, opcao_heartbeat, mostra_menu
carga.c	Gerador automático de clientes para o teste de carga	main, conecta, autentica
Makefile	Compilação	Alvos all (padrão), carga, tudo, clean

O `Makefile` sem parâmetros gera exatamente os dois programas exigidos, `cliente` e `servidor`. O gerador de carga ficou em um alvo separado, `make carga`, justamente para não acrescentar um terceiro executável ao alvo padrão.

Não existem `servidor.h` nem `cliente.h`. Um cabeçalho não faz sentido para um módulo que só contém o `main` e funções privadas ao arquivo, todas declaradas `static`.

2.5 Algoritmos principais

Atendimento de um cliente

Cada thread executa sempre a mesma sequência. Ela autentica a conexão, registra a sessão, processa comandos em laço e limpa tudo ao final.

1. Ler a primeira linha e conferir a credencial. Se falhar, responder `LOGIN_FAIL` e fechar a conexão.
2. Registrar a sessão na tabela de sessões, para o *broadcast* encontrá-la.

3. Em laço: ler uma linha, identificar o comando pela primeira palavra e chamar o tratador correspondente.
4. Ao receber SAIR, ou quando o `recv()` indicar que a conexão caiu, remover a sessão e fechar o socket.

Inserção e *broadcast*

O tratador de ADD valida os dados, insere na fila e avisa os demais. A ordem importa: a inserção acontece antes da notificação, para nenhum cliente ser avisado de um registro que ainda não existe.

1. Separar sequência, identificador e nome da linha recebida.
2. Validar: identificador positivo, nome não vazio e dentro do limite de 49 caracteres.
3. Conferir o número de sequência. Se for repetido, devolver a resposta guardada e encerrar aqui.
4. Inserir na fila, com o *mutex* travado.
5. Gravar no histórico e reescrever o retrato da fila.
6. Enviar o *broadcast* a todas as sessões, exceto à de origem.
7. Responder ADD_OK ao autor e guardar essa resposta.

Heartbeat

O algoritmo compara o marcador da conexão com o tamanho atual da fila. Se o tamanho for maior, o servidor devolve os registros do intervalo e avança o marcador. Se não, devolve ALIVE.

Há um detalhe necessário para a regra funcionar. Quando o próprio cliente faz um ADD, seu marcador avança na hora. Sem isso, o *heartbeat* devolveria a ele o usuário que ele mesmo acabou de cadastrar, o que contraria a definição de “usuários cadastrados por outros clientes”.

2.6 Decisões de implementação

Table 4: Principais decisões tomadas durante a codificação.

Decisão	Motivo
O cliente também usa duas threads: uma lê o socket, a outra lê o teclado	Com um fluxo só, o programa ficaria preso no <code>fgets</code> e a notificação de <i>broadcast</i> só apareceria quando o operador digitasse algo. Isso contraria a exigência de atualização em tempo real. A thread receptora é a única que chama <code>recv()</code>
O texto que trafega na rede é ASCII puro; a acentuação existe só na exibição	O enunciado especifica texto ASCII e o tráfego será comparado com esta documentação. Bytes iguais em qualquer máquina evitam divergência por codificação. O cliente reescreve a mensagem com acento apenas ao mostrá-la na tela

Decisão	Motivo
A fila é lida em lotes de 32 registros ao ser enviada	Mantém o <i>mutex</i> travado por intervalos curtos. Uma listagem longa não impede outros clientes de inserir
O nome é validado uma única vez, no servidor	Corrige um defeito encontrado em teste. O nome era cortado ao entrar na fila, mas a confirmação e o histórico usavam o texto original. Hoje fila, histórico, <i>broadcast</i> e resposta usam exatamente o mesmo texto
O socket vai para a thread dentro de uma <i>struct</i> alocada, não como inteiro convertido em ponteiro	Converter <i>int</i> em <i>void*</i> não é portátil em 64 bits. A <i>struct</i> ainda permite levar o IP de origem e o número da conexão junto
As threads são criadas já desatachadas	Equivale a chamar <i>pthread_detach</i> logo após a criação, com uma chamada a menos por conexão. Faz diferença em 10.000 conexões
O servidor não tem encerramento ordenado	Ele roda até ser interrompido. Como toda gravação é seguida de <i>fflush</i> , nada se perde, e o sistema operacional libera os recursos do processo. Evita código que nunca é executado

O código passou por uma revisão deliberada depois de funcionar. Sete mecanismos que existiam por precaução, e não por necessidade, foram removidos, entre eles o *pthread_rwlock_t* da tabela de sessões e o tratador de SIGINT. O programa saiu de 1.545 para 1.326 linhas de código sem perder nenhuma funcionalidade, e toda a bateria de testes foi repetida com resultados idênticos.

3 Decisões de implementação omissas na especificação

O enunciado exige que qualquer elemento não especificado, mas necessário para o protocolo funcionar, seja descrito de forma explícita. Esta seção é essa declaração. Ela cobre quatro grupos: o que foi acrescentado, o que foi decidido diante de ambiguidade, o que ficou fora de escopo e as limitações conhecidas.

3.1 Elementos acrescentados por necessidade do protocolo

1. **Tabela de sessões** (*sessoes.c*). O *broadcast* exige saber, a cada instante, quais sockets estão conectados e autenticados. Implementada como vetor de tamanho fixo (*MAX_SESSOES* = 12000), protegido por um *mutex*.
2. **Serialização de escrita por socket**. Sem ela, uma mensagem de *broadcast* disparada por outra thread poderia entrar no meio de uma resposta de várias linhas e quebrar o protocolo. Cada sessão tem um *mutex* de envio próprio, mantido travado do cabeçalho ao rodapé do bloco.
3. **Regra para evitar impasse (*deadlock*)**. O *mutex* de envio de uma sessão é obtido sem travar a tabela. Quem o obtém é a própria thread dona da sessão, e só ela remove aquela entrada, então a entrada existe com certeza. A regra que sustenta isso: uma thread nunca remove a própria sessão enquanto segura o *mutex* de envio dela.

4. **Sinal SIGPIPE ignorado**, no cliente e no servidor. Escrever em um socket que o outro lado já fechou gera esse sinal, cujo comportamento padrão é encerrar o processo inteiro. Ignorá-lo transforma a situação em um erro de retorno, que o código já trata.
5. **Pilha reduzida das threads**, de 8 MB para 256 KB, definida por `pthread_attr_setstacksize`. Com o valor padrão, 10.000 threads reservariam 80 GB de espaço de endereçamento e o teste de carga seria impossível.
6. **Numeração das conexões nas linhas de log**. As mensagens começam exatamente com o texto das telas do enunciado e recebem, no fim, o número sequencial e a origem — por exemplo, Novo cliente conectado. [#4271] 192.168.0.20:51344. Sem isso, o print do teste de 10.000 clientes seria uma parede de linhas idênticas, e o enunciado exige que seja possível identificar todas as conexões.

3.2 Pontos ambíguos e como foram resolvidos

Table 5: Pontos em aberto no enunciado e a decisão adotada.

Ponto	Decisão
<i>Heartbeat</i> : texto da Observação contra o <i>print</i>	A Observação diz que a função devolve os usuários cadastrados por outros clientes, ou <i>ALIVE</i> . O <i>print</i> mostra a fila inteira. Seguimos o texto, por ser a especificação escrita
“Atualizar filas em tempo real” no cliente	Implementado como notificação assíncrona do novo usuário, não como réplica local da fila. É o que os <i>prints</i> mostram. O operador pede a fila completa com a opção 2 quando quiser
Terminologia “paciente” contra “usuário”	Adotado “usuário”, termo usado no texto do enunciado
Persistência: banco de dados ou arquivo	Arquivo de texto, opção permitida pelo enunciado. Quatro arquivos, descritos na Seção 3.5
Endereço do servidor	Constante <code>SERVER_IP</code> em <code>comun.h</code> , alterada e recompilada ao mudar de máquina. Nenhum código visto na disciplina faz descoberta dinâmica de rede
Autenticação	Usuário e senha fixos no código, sem validação contra arquivo ou banco. Uma credencial única, coerente com o sistema ter um único tipo de operador
Unicidade de identificador	Não há verificação. Dois usuários podem ter o mesmo identificador, e nada no enunciado exige o contrário
Capacidade da fila	Limite fixo de 20.000 registros. Ao encher, o servidor responde <code>ERRO Fila cheia</code> sem derrubar nada
Tamanho do nome	Até 49 caracteres. Acima disso o cadastro é recusado, não cortado
Identificador inválido	Recusado se for menor ou igual a zero
Recuperação da fila	O servidor sempre inicia com a fila vazia e não recarrega <code>fila.txt</code> . Os arquivos de log e histórico continuam existindo entre execuções

A recusa do nome longo merece uma justificativa. Cortar o nome traria dois problemas: o cliente receberia a confirmação de um nome diferente do que enviou, e o corte em 49

bytes poderia cair no meio de um caractere acentuado, que ocupa dois bytes. Nomes com acento são aceitos normalmente.

3.3 Escopo declarado: o que não foi implementado

A introdução do enunciado cita funcionalidades que não aparecem nas telas, na Observação nem nos oito itens exigidos desta documentação. Elas ficaram fora de escopo, e a decisão está declarada aqui, como o enunciado exige.

Table 6: Funcionalidades citadas apenas na introdução e não implementadas.

Item	Motivo
Painéis administrativos	Citado apenas na introdução. Exigiria um tipo de cliente com interface própria, que não aparece em nenhuma tela
Balanceamento de carga	Pressupõe mais de um servidor. A arquitetura do enunciado mostra um servidor central único
Métricas em tempo real para administradores	Depende do painel administrativo
Papéis distintos de recepcionista, coordenador e administrador	O sistema tem um único tipo de operador, coerente com a credencial única prevista para o login
Posição numérica do usuário na fila	A resposta de LIST segue o formato exato do <i>print</i> (id - nome), sem número de posição. A ordem das linhas já é a ordem de atendimento
Acompanhamento via aplicativo ou página web	O enunciado determina cliente e servidor em C, iniciados por linha de comando

3.4 Limitação conhecida: estouro de inteiro no identificador

O identificador é lido com `sscanf(entrada, "%d", &id)`. Um número acima do limite do tipo `int` sofre estouro silencioso e é gravado com outro valor. Em teste, 9999999999999999 entrou na fila como 276447231.

Identificadores negativos e zero são recusados, mas o estouro produz um valor positivo, então passa pela validação. Corrigir exigiria voltar a `strtoul` com verificação de erro. Como é um comportamento comum em C e sem efeito sobre o funcionamento do sistema, a limitação foi registrada em vez de contornada.

3.5 Persistência em arquivos de texto

O enunciado permite escolher entre banco de dados e arquivo. A escolha foi arquivo de texto, e os dados foram separados em quatro, distinguindo o que é do **cliente** do que é do **usuário**.

Table 7: Arquivos gerados pelo servidor, no subdiretório dados/.

Arquivo	Conteúdo	Quando grava
sessoes.log	LOGIN <ip> <data e hora> e LOGOUT <ip> <data e hora>	A cada entrada e saída de cliente
servidor.log	Os mesmos eventos que aparecem na tela do servidor	A cada evento de conexão
fila.txt	Retrato atual da fila completa	Reescrito a cada ADD
historico.txt	Registro permanente: <data e hora> ADD <id> <nome>	A cada ADD

O arquivo `historico.txt` atende a dois dos itens pedidos pelo enunciado ao mesmo tempo. Cada linha registra um usuário cadastrado com data e hora, servindo tanto como cadastro de usuários quanto como histórico.

4 Retransmissão de mensagens

4.1 O que precisava ser garantido

O TCP garante entrega ordenada e sem perda enquanto a conexão está de pé. Ele garante que os bytes chegaram à outra máquina, mas não que a aplicação do outro lado processou o pedido. Se o servidor estiver vivo e travado, ou se a conexão cair no meio de uma operação, o TCP não resolve o problema do cliente.

A retransmissão implementada é, portanto, do nível da aplicação. Ela responde a duas perguntas: como o cliente sabe que seu comando foi processado, e o que ele faz quando não sabe.

4.2 Mecanismo adotado

O mecanismo tem três partes, e nenhuma delas usa uma mensagem de confirmação separada.

1. A resposta é a confirmação. Toda resposta listada na Seção 2.3 funciona como confirmação do pedido correspondente. O `ADD_OK` confirma que o `ADD` foi recebido e processado. Não existe uma mensagem `ACK` à parte.

2. Tempo limite e reenvio. Depois de enviar um comando, o cliente espera a resposta por até **3 segundos**. Se o prazo estourar, ele reenvia o mesmo comando, até um total de **3 tentativas**. Se as três falharem, avisa o operador com `Servidor não respondeu`, tente novamente. e volta ao menu.

3. Número de sequência e idempotência. Cada comando carrega um número de sequência próprio daquela conexão. O servidor guarda o último número processado. Se receber um número repetido, ele devolve a mesma resposta de antes **sem processar de novo**. É isso que impede um reenvio de cadastrar o mesmo usuário duas vezes.

Do lado do cliente há uma regra complementar. Uma resposta só é exibida se houver um comando esperando por ela. Cópias atrasadas, que chegam depois de o cliente ter desistido, são descartadas em silêncio. Sem essa regra, uma cópia atrasada seria confundida com

a resposta do comando seguinte.

4.3 Avaliação honesta do mecanismo

Sobre uma única conexão TCP, o reenvio é em boa parte redundante, porque o próprio TCP já retransmite segmentos perdidos. Vale reconhecer isso.

O que de fato agrega é o **número de sequência**. Ele é o que torna a operação idempotente e impede que um comando repetido produza efeito duplicado. Essa garantia o TCP não oferece, porque ela é sobre o processamento na aplicação, não sobre a entrega dos bytes.

5 Testes e análise

5.1 Como os testes foram feitos

Os testes seguiram três frentes, todas repetidas a cada alteração do código:

1. **Teste funcional** — dois clientes reproduzindo a sequência das telas do enunciado.
2. **Casos especiais** — um programa em Python conversando direto com o servidor pelo socket, enviando comandos válidos e inválidos e conferindo cada resposta. Falar o protocolo em texto permite testar situações que o `./cliente` não deixaria acontecer, como um comando malformatado ou um comando antes do login.
3. **Carga** — o programa `carga.c`, com 100, 1.000 e 10.000 clientes.

5.2 Teste funcional

Table 8: Teste funcional com dois clientes conectados ao mesmo servidor.

#	Ação	Resultado esperado	Obtido
1	Clientes A e B conectam	Conectado ao servidor. e Servidor: LOGIN_OK	OK
2	A cadastra 10/Abel e 30/Breno	Usuário Abel adicionado.	OK
3	B recebe as notificações	[Broadcast] Novo usuário: Abel e Breno	OK
4	B cadastra 20/Carlos e 40/Dario	Confirmação em B, <i>broadcast</i> em A	OK
5	A escolhe 2	Os quatro usuários, entre cabeçalho e rodapé	OK
6	A escolhe 3	Só 20 - Carlos e 40 - Dario, cadastrados por B	OK
7	B escolhe 3	Heartbeat: ALIVE, pois B só cadas- trou os próprios	OK
8	Ambos escolhem 0	Cliente desconectado. na tela do servidor	OK

O passo 7 confere com o *print* do enunciado, que também mostra ALIVE para o cliente que acabou de cadastrar. O passo 6 é onde seguimos o texto da Observação em vez do *print*: o *print* mostra a fila inteira, e o sistema devolve apenas os usuários de outros clientes.

5.3 Casos especiais: 30 de 30 aprovados

Foram testados 30 casos especiais, agrupados por tema. Todos passaram. Em todos os casos de comando inválido, **a conexão permanece aberta** — um erro do operador não derruba o cliente.

Table 9: Casos especiais testados e resposta obtida.

#	Situação testada	Resposta obtida
<i>Autenticação</i>		
1	Senha errada	LOGIN_FAIL
2	Conexão após falha de login	Encerrada pelo servidor
3	LOGIN sem a senha	LOGIN_FAIL
4	Comando antes do login	LOGIN_FAIL
5	Credencial correta	LOGIN_OK
<i>Comandos malformados</i>		
6	Comando inexistente	ERRO Comando desconhecido
7	ADD sem argumento	ERRO Formato esperado...
8	ADD só com a sequência	ERRO Formato esperado...
9	ADD com identificador não numérico	ERRO Formato esperado...
10	ADD sem o nome	ERRO Nome invalido...
11	LIST sem número de sequência	ERRO Formato esperado...
12	HEARTBEAT sem número de sequência	ERRO Formato esperado...
13	Linha em branco	Ignorada; a conexão segue
<i>Validação de dados</i>		
14	Identificador negativo	ERRO 0 identificador deve ser um numero positivo
15	Identificador zero	ERRO ...numero positivo
16	Nome com 60 caracteres	ERRO Nome invalido, vazio ou longo demais
17	Nome com espaços	Aceito
18	Nome com acentos	Aceito
<i>Retransmissão, lado do servidor</i>		
19	ADD repetido com o mesmo número de sequência	Mesma resposta ADD_OK 10 Abel
20	Fila após o ADD repetido	Um único registro; não duplicou
21	Sequência menor que a última processada	ERRO Numero de sequencia invalido
22	Sequência zero	ERRO Numero de sequencia invalido
<i>Heartbeat e broadcast</i>		
23	<i>Broadcast</i> chega ao outro cliente	[Broadcast] Novo usuario: Breno
24	<i>Heartbeat</i> com novidade	Só 30 - Breno, cadastrado por outro
25	O mesmo <i>heartbeat</i> reenviado	Bloco idêntico ao anterior
26	<i>Heartbeat</i> seguinte, sem novidade	ALIVE
27	Esse <i>heartbeat</i> reenviado	ALIVE de novo
28	Novo cadastro e novo <i>heartbeat</i>	Só 40 - Dario, o registro novo

#	Situação testada	Resposta obtida
29	Esse <i>heartbeat</i> reenviado <i>Falha de conexão</i>	Mesmo bloco
30	Cliente cai sem enviar SAIR	O servidor detecta, remove a sessão e continua atendendo os demais

Análise. Os casos 25, 27 e 29 são os mais importantes do conjunto. Eles comprovam a idempotência: reenviar um comando devolve exatamente a mesma resposta e não avança o marcador do que já foi visto. Sem essa propriedade, o mecanismo de retransmissão da Seção 4 seria inseguro, porque cada reenvio produziria um efeito novo.

O caso 30 comprova que a queda abrupta de um cliente é tratada como um evento normal. O `recv()` devolve zero quando o outro lado fecha, e a thread sai do laço, remove a sessão e fecha o socket, sem afetar as demais.

5.4 Retransmissão vista do lado do cliente

O comportamento do cliente foi testado contra um servidor de mentira, que aceita o login e depois não responde a mais nada. O objetivo era forçar o tempo limite.

```
t = 0,0s   servidor recebeu: ADD 1 10 Abel
t = 3,0s   servidor recebeu: ADD 1 10 Abel
t = 6,0s   servidor recebeu: ADD 1 10 Abel
```

Figure 2: Registro do servidor de mentira durante o teste de retransmissão.

Os resultados confirmaram o projetado:

- exatamente 3 envios, um a cada 3 segundos;
- os três com o mesmo número de sequência, que é o que permite ao servidor reconhecer o reenvio;
- o cliente avisou o operador ao fim das tentativas;
- o servidor de mentira enviou depois 3 cópias atrasadas da resposta, e o cliente ignorou todas, porque já não havia comando esperando;
- o comando seguinte funcionou normalmente, sem confusão.

5.5 Compilação

Table 10: Verificações de compilação.

Verificação	Resultado
make sem parâmetros	Gera exatamente cliente e servidor
Avisos com -Wall -Wextra	Nenhum
Avisos com -Wpedantic -Wshadow	Nenhum
-Wconversion -Wsign-conversion	
-Wcast-align -Wnull-dereference	
Compilação com -std=c99, -std=c11 e -std=c17, com e sem -O2	Sem erro e sem aviso nas oito combinações

6 Teste de carga com 100, 1.000 e 10.000 clientes

6.1 Como a carga foi gerada

A geração é automática, feita pelo programa `carga.c` — o segundo código cliente que o enunciado permite expressamente. Ele repete `socket()`, `connect()` e `login` N vezes, e recebe a quantidade como parâmetro:

```
make carga
./carga 100
./carga 1000
./carga 10000
```

Listing 3: Comandos usados no teste de carga.

Duas providências foram necessárias para o teste medir o que devia medir. A primeira: as conexões ficam **todas abertas ao mesmo tempo** até o operador pressionar ENTER. Sem isso, o programa estaria abrindo e fechando conexões em sequência, e o servidor nunca precisaria sustentar mais de uma por vez. A segunda: cada cliente gerado **também se autentica**, ocupando uma sessão no servidor exatamente como um `./cliente`.

O servidor foi reiniciado antes de cada teste, para a numeração das conexões começar em `[#1]`. Antes do teste de 10.000, o limite de descritores de arquivo do terminal foi elevado com `ulimit -n 20000`.

6.2 Resultados

Table 11: Resultado dos três testes de carga.

Clientes	Conexões abertas	Falhas	Numeração no servidor	Sessões autenticadas
100	100	0	<code>[#1]</code> a <code>[#100]</code>	100
1.000	1.000	0	<code>[#1]</code> a <code>[#1000]</code>	1.000
10.000	10.000	0	<code>[#1]</code> a <code>[#10000]</code>	10.000

Depois de cada teste, um cliente comum foi executado e operou normalmente. Isso mostra que o servidor não ficou em estado ruim após sustentar a carga.

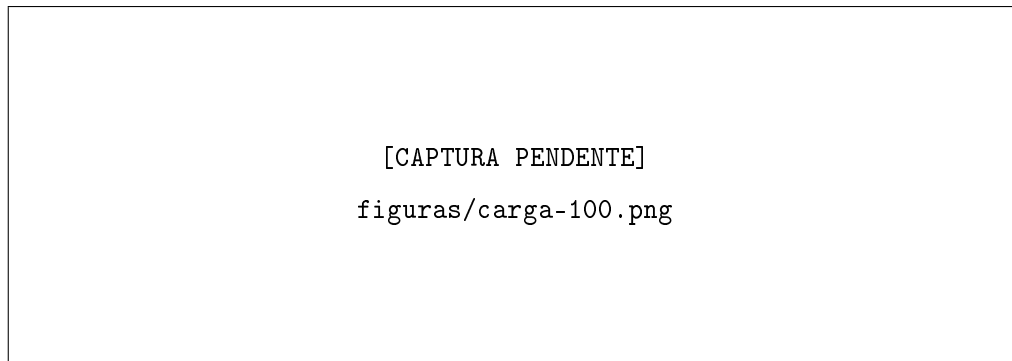


Figure 3: Tela do servidor durante o teste com 100 clientes, com as conexões numeradas de [#1] a [#100].

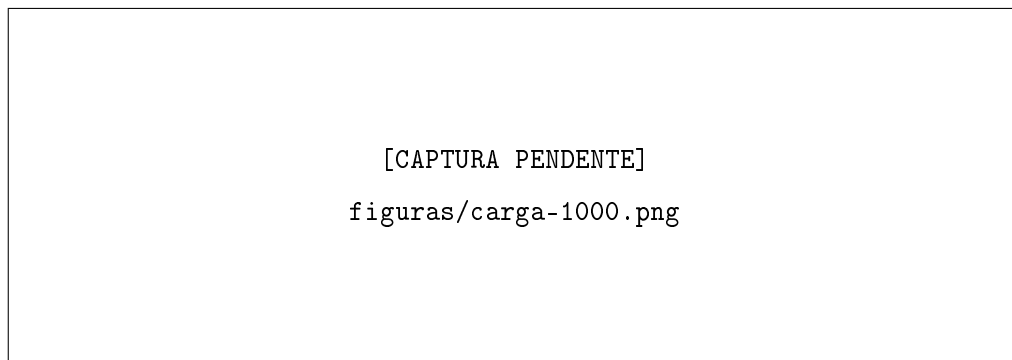


Figure 4: Tela do servidor durante o teste com 1.000 clientes.

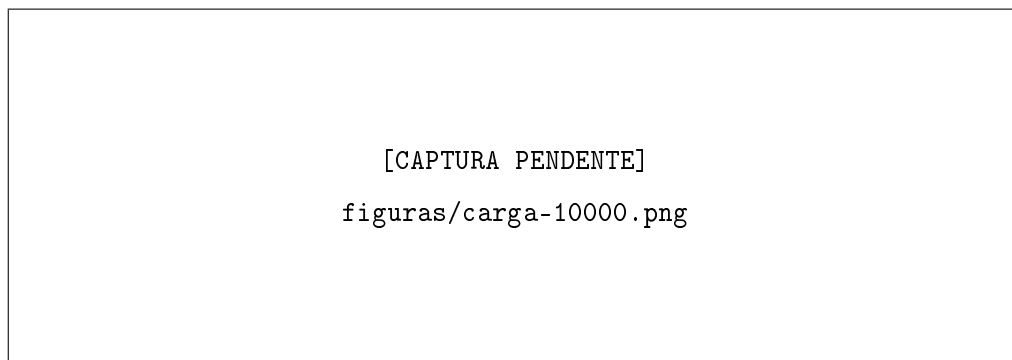


Figure 5: Tela do servidor durante o teste com 10.000 clientes.

6.3 Análise dos resultados

Por que 10.000 threads couberam. Cada thread é criada com pilha de 256 KB, definida por `pthread_attr_setstacksize`. Com o padrão do sistema, de 8 MB, 10.000 threads reservariam 80 GB de espaço de endereçamento e o teste seria impossível. Com 256 KB, o total fica em torno de 2,5 GB de memória virtual, que é espaço reservado e não ocupado fisicamente.

Por que as linhas não saem em ordem numérica perfeita. No teste de 100 clientes, a linha [#99] apareceu depois da [#100]. Isso não é defeito. Cada conexão é atendida por uma thread diferente, e a ordem em que elas escrevem no terminal depende de qual thread o sistema escalona primeiro. O número é atribuído no momento da aceitação, então a contagem está correta. Na prática, é uma evidência visual de que o servidor é mesmo concorrente.

7 Telas do funcionamento correto

Esta seção mostra o sistema em operação. As capturas seguem a mesma sequência do teste funcional descrito na seção anterior.

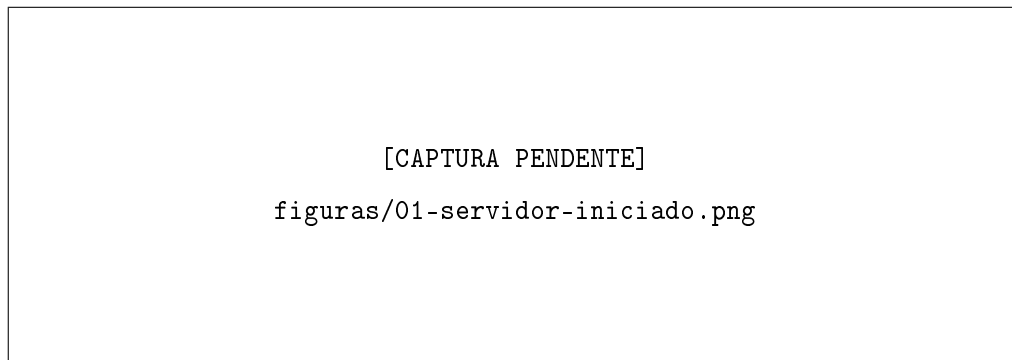


Figure 6: Servidor recém-iniciado, mostrando Servidor iniciado na porta 8080 e as conexões dos dois clientes.

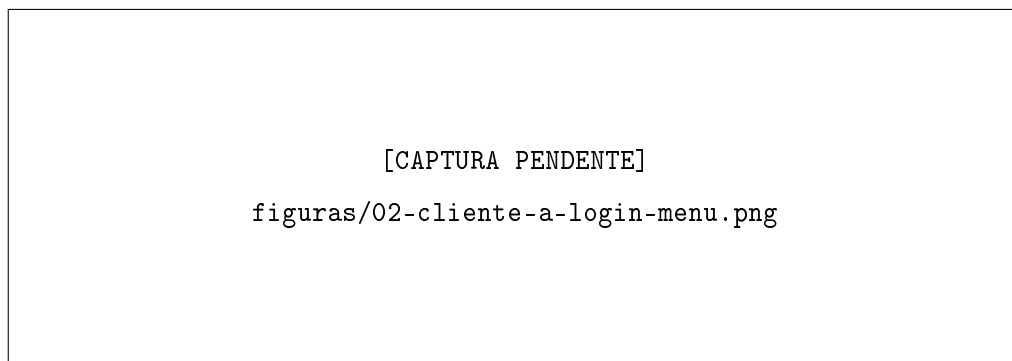


Figure 7: Cliente A conectado e autenticado, com o menu de operação.

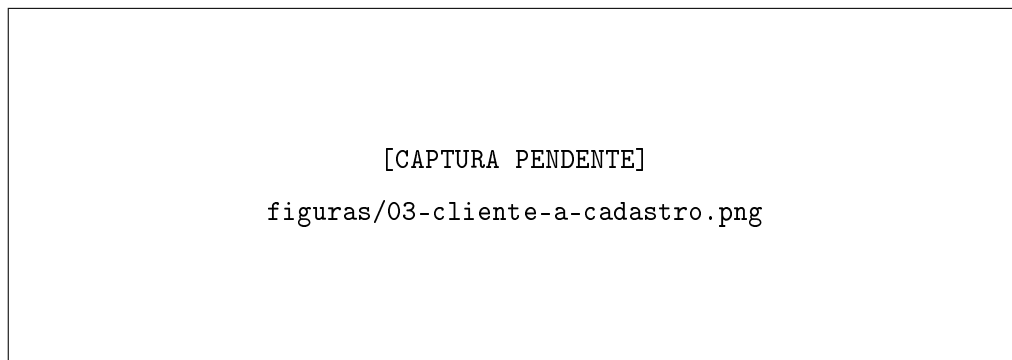


Figure 8: Cliente A cadastrando os usuários 10 - Abel e 30 - Breno.

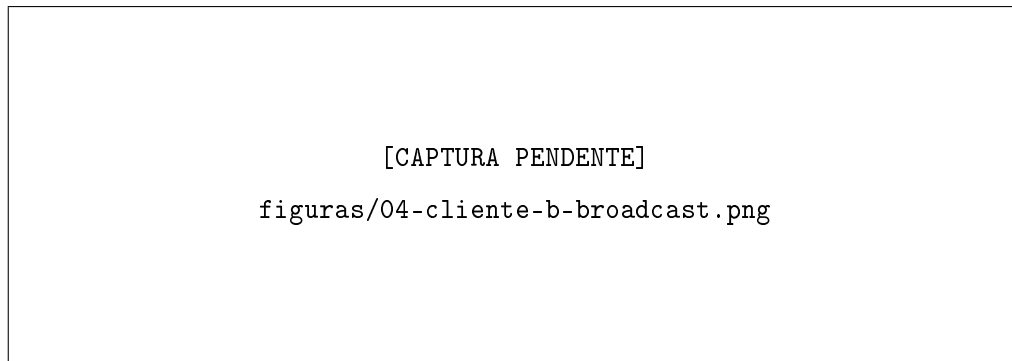


Figure 9: Cliente B recebendo a notificação de *broadcast* sem ter pedido nada. Esta é a evidência visual do funcionamento em tempo real.

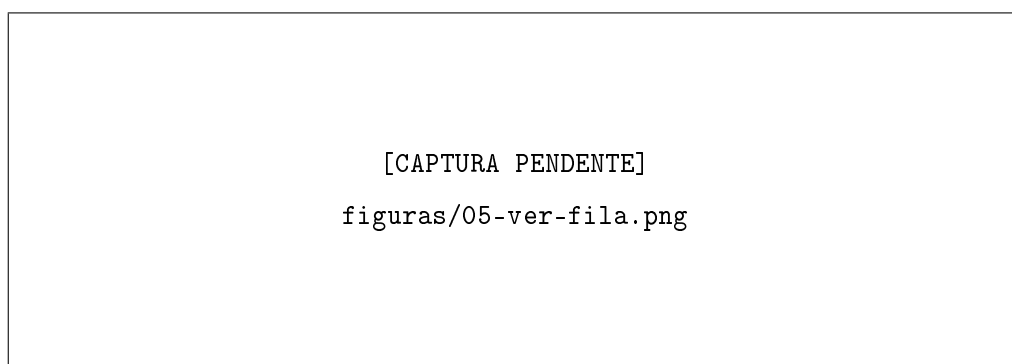


Figure 10: Opção 2, com a fila completa entre o cabeçalho e o rodapé.

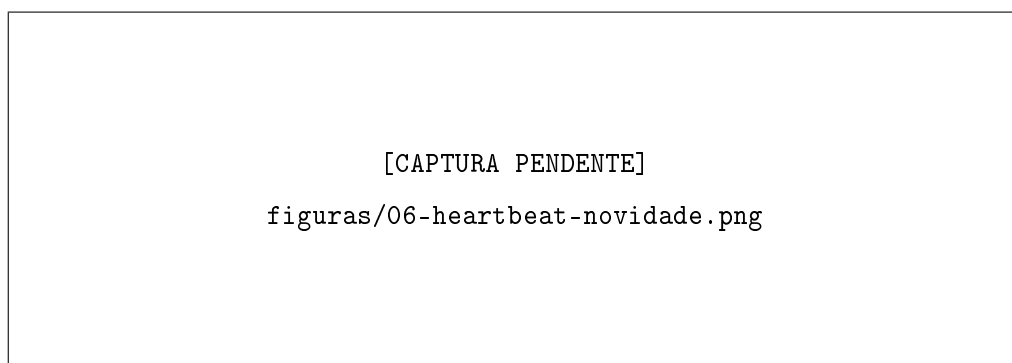


Figure 11: Opção 3 no cliente A: apenas os usuários cadastrados pelo cliente B.

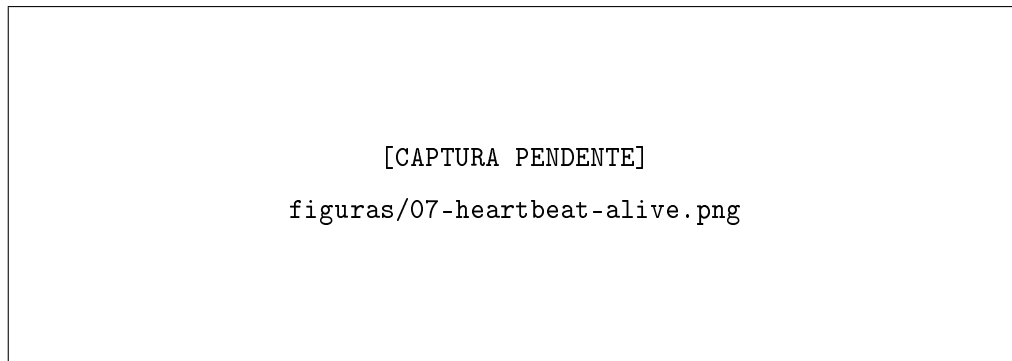


Figure 12: Opção 3 repetida, sem novidade desde a última verificação: resposta ALIVE.

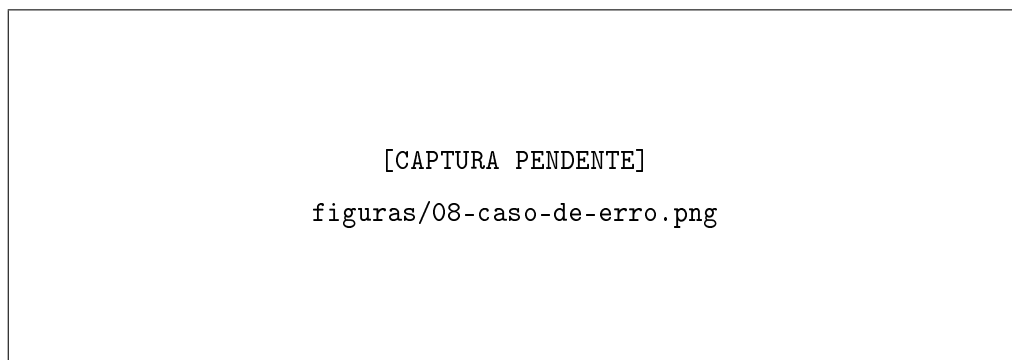


Figure 13: Tratamento de erro: cadastro recusado por identificador inválido. A conexão permanece aberta.

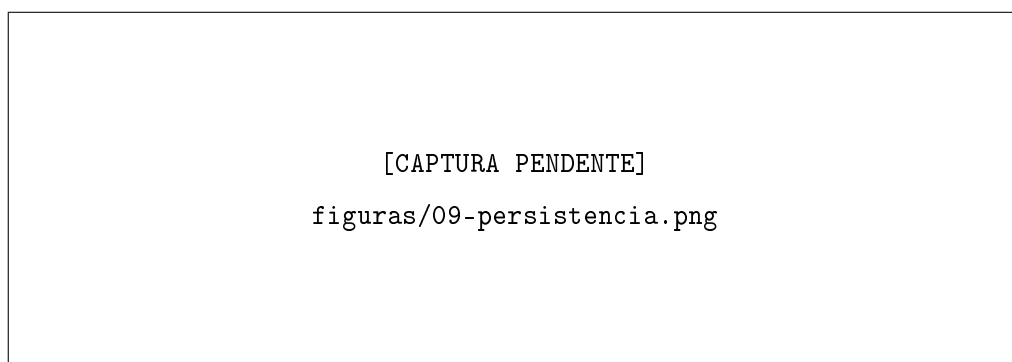


Figure 14: Conteúdo de dados/fila.txt e dados/historico.txt, gerados pelo servidor.

8 Conclusão

O sistema atende ao que o enunciado especifica. Cliente e servidor reproduzem as telas de exemplo ponta a ponta, o *heartbeat* segue a Observação, a persistência grava os cinco tipos de dado pedidos e o servidor sustentou 10.000 conexões simultâneas autenticadas sem nenhuma falha. O código final tem 1.326 linhas em seis módulos e compila sem nenhum aviso.

Mais interessante que o resultado, porém, é o caminho até ele. Quatro dificuldades marcaram o desenvolvimento.

A delimitação de mensagens. Nenhum dos códigos vistos em aula precisa resolver esse problema, porque cada um troca uma única mensagem por conexão. Aqui a conexão é persistente e carrega dezenas de mensagens, e o TCP não separa uma da outra. Foi preciso projetar do zero a convenção de uma linha por mensagem e o buffer que junta os pedaços recebidos. Esse foi o ponto do trabalho que mais exigiu leitura fora do material da disciplina.

A escolha entre `fork()` e `threads`. O `fork()` é a técnica mais detalhada em aula, e a primeira escolha natural. O problema apareceu ao pensar na fila: processos têm memória isolada, e compartilhar a fila exigiria comunicação entre processos, que não foi ensinada. As `threads` resolvem o compartilhamento sozinhas e cobram o *mutex* em troca. Foi uma troca consciente, não uma preferência.

O teste de 10.000 clientes não funcionou de primeira. A primeira tentativa falhou por falta de memória. A causa levou algum tempo para aparecer: cada thread reserva 8 MB de pilha por padrão, o que daria 80 GB. A solução foi reduzir a pilha para 256 KB. Esse foi o momento em que ficou claro que “suportar N clientes” depende de detalhes que não estão no código do protocolo.

Dois defeitos apareceram nos próprios testes. O primeiro: o nome era cortado ao entrar na fila, mas a confirmação e o histórico usavam o texto original, então o cliente recebia a confirmação de um nome diferente do que enviou. Passou a ser recusado em vez de cortado. O segundo: as conexões do teste de carga apareciam como falha de autenticação quando, na verdade, encerravam antes de tentar logar. São situações diferentes e passaram a ser registradas como tais. Nenhum dos dois teria sido encontrado sem os casos especiais.

O código foi enxugado depois de funcionar. Sete mecanismos que existiam por precaução, e não por necessidade, foram removidos após a primeira versão passar em todos os testes. O programa saiu de 1.545 para 1.326 linhas sem perder nenhuma funcionalidade, e a bateria de testes foi repetida por inteiro com resultados idênticos. O critério foi manter apenas o que o enunciado exige ou o que o protocolo precisa para funcionar.

Uma ressalva sobre a retransmissão. Sobre uma conexão TCP única, o reenvio de comandos é em boa parte redundante, porque o TCP já retransmite segmentos perdidos. O que de fato agrega valor é o número de sequência, que impede um comando repetido de cadastrar o mesmo usuário duas vezes. Reconhecer isso parece mais correto do que apresentar o mecanismo como indispensável.

O trabalho deixa caminhos abertos. Ler o endereço do servidor de um arquivo de configuração, em vez de recompilar, é a melhoria mais imediata. Substituir o vetor fixo por uma estrutura dinâmica removeria o limite de 20.000 registros. E migrar de uma thread por conexão para multiplexação com `select()` reduziria o custo por cliente, ao preço de um código bem mais complexo.

Referências bibliográficas

References

- [1] FREE SOFTWARE FOUNDATION. *GNU Make Manual*. Disponível em:
<http://www.gnu.org/software/make/manual/make.html>. Acesso em: 28 jul.

2026.

- [2] COLBY COLLEGE. *Makefile Tutorial*. Disponível em:
<http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/>. Acesso em: 28 jul. 2026.
- [3] THE LINUX MAN-PAGES PROJECT. *Linux Programmer's Manual*. Páginas consultadas: `socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `send(2)`, `recv(2)`, `setsockopt(2)`, `pthread_create(3)`, `pthread_mutex_init(3)`, `pthread_cond_timedwait(3)`, `pthread_attr_setstacksize(3)`, `sscanf(3)`. Disponível em: <https://man7.org/linux/man-pages/>. Acesso em: 28 jul. 2026.
- [4] FERREIRA, Rubens Barbosa. *Códigos de exemplo da disciplina de Redes de Computadores*: `bind.c`, `escrevendo.c`, `fork.c`, `multithread.c`, `porta.c`, `redes.c`. Dourados: Universidade Estadual de Mato Grosso do Sul, 2026.